

TechDebt AI

Technical Documentation

Table of Contents

Table of Contents	2
1. System Overview	3
2. Architecture	3
2.1 Request flow	3
2.2 Service topology	3
3. Data Model	4
4. Backend Structure	4
5. API Reference	4
5.1 Configuration	5
5.2 Dashboard	5
5.3 SonarQube	5
5.4 GitHub	5
5.5 LLM analysis (Bedrock + Ollama)	5
5.6 Machine learning	5
5.7 Observability & scanning	5
6. Machine Learning Pipeline	7
6.1 Feature extraction	7
6.2 Labelling	7
6.3 Training & evaluation	7
6.4 Experiment tracking	7
7. Frontend Structure	8
8. Deployment and Configuration	8
8.1 Environment variables	8
8.2 Bring the stack up	8
9. LLM Providers	9
10. Observability	9
10.1 MLflow	9
10.2 Langfuse	9
11. Operations Runbook	9
11.1 Common commands	9
12. Security Considerations	9

1. System Overview

TechDebt AI is a multi-service application orchestrated with Docker Compose. A Vue 3 single page app (served by Nginx) calls a FastAPI backend, which aggregates data from GitHub, SonarQube, AWS Bedrock, a local Ollama model, MLflow and Langfuse. A single Postgres instance backs SonarQube, MLflow, Langfuse and the application.

The backend mounts source as a volume with hot-reload, so Python changes take effect without a rebuild. The frontend is a static Nginx build and must be rebuilt to pick up changes.

2. Architecture

2.1 Request flow

```
Browser
|
Nginx (:80) ----- serves Vue SPA (static build)
| /api/* (proxy, 300s read timeout)
FastAPI backend (:8000)
|-- GitHub REST API      (repo, commits, churn, PRs)
|-- SonarQube REST API    (:9000 metrics, issues, component tree)
|-- AWS Bedrock           (Nova Micro/Lite/Pro, Mistral)
|-- Ollama                (:11434 llama3.2:3b, local)
|-- MLflow HTTP API       (:5000 experiment tracking)
|-- Langfuse HTTP API     (:3000 LLM observability)
|
PostgreSQL (databases: sonarqube, mlflow, langfuse, appdb)
```

2.2 Service topology

Service	Image	Port (host)	Responsibility
frontend	nginx (custom build)	80	Serves the Vue SPA, proxies /api to backend
backend	python/uvicorn	8000	FastAPI application & integration layer
sonarqube	sonarqube:10.7 CE	9000	Static analysis engine & issue store
mlflow	mlflow 2.13	5001->5000	Experiment tracking UI/API
langfuse	langfuse 2	3001->3000	LLM tracing UI/API
ollama	ollama/ollama	11434	Local LLM inference (llama3.2:3b)
ollama-init	ollama/ollama	-	job that pulls the local model
postgres	postgres:16	5432 (internal)	Shared database for all services
scanner	sonar-scanner-cli:5	-	On demand scan job

Service	Image	Port (host)	Responsibility
	(+JDK/Maven)		

3. Data Model

A single PostgreSQL instance hosts four logical databases, created on first start by `scripts/init-db.sh`:

Database	Owner / purpose
sonarqube	SonarQube's own schema (projects, issues, measures, snapshots)
mlflow	MLflow tracking store (experiments, runs, params, metrics, tags)
langfuse	Langfuse traces, observations, and scores
appdb	Application database

4. Backend Structure

The backend follows a layered layout under `backend/app/`

Package	Contents
<code>routers/</code>	One module per API domain, each mounts an <code>/api/<domain></code> prefix
<code>services/</code>	Thin async clients for each external system (sonar, github, bedrock, ollama) plus <code>risk_scorer</code> and <code>ml_scorer</code>
<code>models/</code>	request/response schemas
<code>config.py</code>	Settings loaded from environment / <code>.env</code>
<code>state.py</code>	<code>AVAILABLE_REPOS</code> , <code>PR_ENABLED_REPOS</code> , active-repo accessor
<code>main.py</code>	FastAPI app, registers all routers, <code>/health</code> and <code>/</code> endpoints

5. API Reference

All endpoints are prefixed with `/api` and returned as JSON. Base URL in the stack is `http://localhost:8000` (proxied at `http://localhost/api`).

5.1 Configuration

GET `/api/config/repo` - Return the currently active repository and the list of available repos.

POST `/api/config/repo` - Switch the active repository. Body: `{ repo: "owner/name" }`.

5.2 Dashboard

GET `/api/dashboard/summary` - Aggregated repo metadata + SonarQube metrics + rule-based risk score, logs an MLflow run.

GET `/api/dashboard/churn` - Per-file commit churn cross referenced with SonarQube components, sorted by churn.

5.3 SonarQube

GET `/api/sonar/metrics` - Project level measures for the active repo.

GET `/api/sonar/issues` - Issues with severity/type/tag facets. Query: `page_size`.

GET `/api/sonar/hotspots` - Security hotspots for the project.

GET `/api/sonar/history` - Historical measure series. Query: `metric`.

GET `/api/sonar/components` - Per-file metrics for all files (paginated internally).

5.4 GitHub

GET `/api/github/repo` - Repository metadata.

GET `/api/github/commits` - Recent commits. Query: `per_page`.

GET `/api/github/languages` - Language byte breakdown.

GET `/api/github/contributors` - Contributor list.

POST `/api/github/create-fix-pr` - LLM generate a fix for an issue and open a pull request. Traced in Langfuse.

5.5 LLM analysis (Bedrock + Ollama)

GET `/api/bedrock/models` - List available Bedrock model aliases.

POST `/api/bedrock/analyse` - Repo level debt analysis. Body: `{ model_alias }`.

POST `/api/bedrock/analyse-issue` - Explain a single issue and propose a fix.

POST `/api/bedrock/compare` - Run the same prompt through a Bedrock model and Ollama side by side.

5.6 Machine learning

POST `/api/ml/train` - Train LR + RF on file metrics for the active repo. Returns CV metrics, ROC curve points, feature importances, logs to MLflow.

GET `/api/ml/predict` - Per-file predictions (rule based + LR + RF probabilities), sorted by combined probability.

5.7 Observability & scanning

GET `/api/mlflow/experiments` - List MLflow experiments.

GET `/api/mlflow/runs` - List MLflow runs. Query: `max_results`, `page_token`.

GET `/api/langfuse/traces` - List Langfuse traces. Query: `limit`, `page`.

GET `/api/ollama/status` - Local model availability and installed models.

POST `/api/ollama/generate` - Direct generation against the local model.

POST `/api/scan/trigger` - Trigger a SonarQube scan job.

GET `/api/scan/status` - Scan job status.

GET `/health` - Liveness probe `{ status: "ok" }`.

6. Machine Learning Pipeline

6.1 Feature extraction

`ml_scorer._components_to_df()` converts the SonarQube component tree into a DataFrame. Each row is one file with the eight model features, a rule based file score, and a derived binary label.

Feature	Source	Type
bugs / vulnerabilities / code_smells	SonarQube issue counts	numeric
cognitive_complexity	SonarQube	numeric
duplicated_lines_density	SonarQube (%)	numeric
sqale_rating	SonarQube (1–5)	numeric
churn	GitHub commit count per file	numeric
severity_score	Weighted violation sum, normalised	numeric 0–1

6.2 Labelling

`label = 1 if compute_file_risk_score(metrics) >= 20 else 0`. The file level scorer uses tight ceilings (bugs 5, vulns 3, smells 20, complexity 50) plus a severity term (BLOCKERx5 ... INFOx1, ceiling 10).

6.3 Training & evaluation

```
lr = Pipeline([('scaler', StandardScaler()),
               ('clf', LogisticRegression(max_iter=1000,
                                         class_weight='balanced'))])
rf = RandomForestClassifier(n_estimators=100,
                           class_weight='balanced', random_state=42)

cv = StratifiedKFold(n_splits=min(5, ...), shuffle=True, random_state=42)
y_pred = cross_val_predict(clf, X, y, cv=cv, method='predict')
y_proba = cross_val_predict(clf, X, y, cv=cv, method='predict_proba')[ :,1]
```

Single-class repositories fall back to a `DummyClassifier(most_frequent)` and report null metrics rather than misleading perfect scores. Non-finite ROC thresholds (inf) are sanitised to null for JSON safety.

6.4 Experiment tracking

- Experiment `technical-debt-scoring` - one run per dashboard summary (rule-based score + raw metrics).
- Experiment `ml-risk-scoring` - one run per model per training call, logging params (features, threshold, sample counts), metrics (precision/recall/F1/AUC), and artifacts (`feature_importances.json`, ROC points).

7. Frontend Structure

The Vue 3 app under `frontend/src/` uses Vite, Pinia, Axios, and Chart.js:

Path	Role
api/index.js	Central Axios instance, every backend call is defined here
stores/	Pinia stores for dashboard and AI analysis state
components/	Feature SFCs: RiskHeatmap, BedrockPanel, RiskBreakdown, DebtTrendChart, etc.
App.vue	Root, navigation and panel composition

8. Deployment and Configuration

8.1 Environment variables

Variable	Purpose
AWS_ACCESS_KEY_ID / _SECRET_ACCESS_KEY	Bedrock credentials
AWS_DEFAULT_REGION	Bedrock region (EU model IDs default)
GITHUB_TOKEN	repo:read (+ write for PR creation)
SONARQUBE_TOKEN	Generated in the SonarQube UI after first start
LANGFUSE_PUBLIC_KEY / _SECRET_KEY	Generated in the Langfuse UI
TARGET_REPO	Default active repository
SSL_VERIFY	Set false to bypass httpx SSL verification (corporate proxies)

8.2 Bring the stack up

```
cp .env.example .env      # then fill in the keys above
docker compose up -d      # start all services
# generate SonarQube + Langfuse tokens in their UIs, add to .env
docker compose restart backend
docker compose --profile scan run --rm scanner  # run a scan
```

9. LLM Providers

Alias	Model ID	Notes
nova-micro	eu.amazon.nova-micro-v1:0	Cheapest Bedrock tier
nova-lite	eu.amazon.nova-lite-v1:0	Default
nova-pro	eu.amazon.nova-pro-v1:0	Most capable Nova
mistral	mistral.mixtral-8x7b-instruct-v0:1	Bedrock Mistral
ollama	ollama/llama3.2:3b	Local, zero-cost, private

10. Observability

10.1 MLflow

- UI at <http://localhost:5001>. Two experiments track rule-based and ML runs. Open a run to view params, metrics and downloadable artifacts.

10.2 Langfuse

- UI at <http://localhost:3001>. Every LLM call is a trace with a nested generation. Trace input carries the request context. The generation carries prompt, output, model id and token/latency data. Failed calls are recorded at ERROR level.

11. Operations Runbook

11.1 Common commands

```

docker compose ps                # service status
docker compose logs -f backend   # tail backend logs
docker compose build frontend && \
  docker compose up -d frontend  # apply frontend changes
docker compose --profile scan run --rm scanner # rescan active repo
docker compose down              # stop (keeps volumes/data)
docker compose down -v          # stop and WIPE all data

```

12. Security Considerations

- Secrets (AWS, GitHub, SonarQube, Langfuse) are supplied via `.env` and never committed, `.env.example` documents the shape only.
- `SSL_VERIFY=false` exists only to traverse SSL-inspecting corporate proxies in a demo context, it should remain true in any real deployment.
- CORS is currently open (`allow_origins=["*"]`) for local development and should be restricted before exposure.
- LLM-generated fixes are raised as pull requests for human review, never merged automatically.

- GitHub PR creation is limited to repositories in `PR_ENABLED_REPOS` where the token has write access.